

Laboratorio de Patrones y Agregación: MongoDB

Departamento de Informática

22 de abril de 2026

1 Introducción al Laboratorio

En MongoDB, la potencia no reside en guardar documentos, sino en cómo podemos transformarlos y analizarlos en tiempo real. Este laboratorio se centra en el **Aggregation Framework**, la herramienta más potente de MongoDB para el análisis de datos masivos.

1.1 Patrón 1: Consultas Polimórficas

Escenario: Queremos encontrar solo aquellas experiencias que tienen requerimientos técnicos específicos (como la profundidad en buceo o menús en gastronomía).

Filtrado por existencia de campos

```
// Buscar experiencias de Aventura que tengan el campo 'profundidad'
db.experiencias.find({
  tipo: "Aventura",
  "detalles.profundidad": { $exists: true }
})
```

Reflexión Pedagógica: Fíjate en el uso del «Dot Notation» (`detalles.profundidad`) para acceder a campos dentro de objetos anidados. En SQL, esto requeriría un JOIN o una tabla de atributos muy compleja.

1.2 Patrón 2: El Pipeline de Agregación (La Tubería)

Escenario: El departamento de Marketing quiere un informe de las categorías de experiencias más rentables (precio medio > 50€).

Prueba a ejecutar este pipeline por etapas para ir entendiendo qué hace cada cosa.

Pipeline Complejo

```
db.experiencias.aggregate([
  // Etapa 1: Filtrar documentos iniciales
  { $match: { precio: { $gt: 0 } } },

  // Etapa 2: Agrupar por tipo y calcular métricas
  { $group: {
    _id: "$tipo",
    precio_medio: { $avg: "$precio" },
    cantidad: { $sum: 1 }
  } },

  // Etapa 3: Filtrar los resultados del grupo (el 'HAVING' de SQL)
  { $match: { precio_medio: { $gt: 50 } } },

  // Etapa 4: Ordenar y proyectar campos limpios
  { $sort: { precio_medio: -1 } },
  { $project: { _id: 0, categoria: "$_id", precio_medio: 1, cantidad: 1 } }
])
```

1.3 Patrón 3: Análisis de Arrays con \$unwind

Escenario: Queremos saber cuál es el **tag** (etiqueta) más popular en todo nuestro catálogo.

Contar elementos dentro de arrays

```
db.experiencias.aggregate([
  { $unwind: "$tags" }, // "Rompe" el array en documentos individuales
  { $group: {
    _id: "$tags",
    total: { $sum: 1 }
  } },
  { $sort: { total: -1 } },
  { $limit: 5 } // Ver el Top 5
])
```

Análítica: Si una experiencia tiene 3 tags, **\$unwind** genera 3 copias temporales del documento. **¡Ojo!** Por defecto, si un documento no tiene el campo **tags** o está vacío, **desaparecerá del pipeline**. Para evitarlo, se usa el parámetro **preserveNullAndEmptyArrays: true**.

1.4 Laboratorio de Depuración: Tipos de Datos

MongoDB es flexible, pero no mágico. El número 100 y el texto "100" son tipos distintos y las comparaciones numéricas fallarán.

Peligro: Comparación de Tipos

```
// 1. Insertamos un dato con precio como TEXTO (error comun)
db.experiencias.insertOne({
  nombre: "Error de Tipo",
  precio: "300" // Notese las comillas
})

// 2. Intentamos buscar experiencias de mas de 100
db.experiencias.find({ precio: { $gt: 240 } })
// RESULTADO: El documento "Error de Tipo" NO aparecera.
```

1.4.1 Detección con \$type

Para limpiar tu base de datos, puedes buscar qué documentos tienen un campo con un tipo erróneo. El tipo «string» en MongoDB es el número 2.

```
// Buscar documentos donde 'precio' sea un String (tipo 2)
db.experiencias.find({ precio: { $type: 2 } })

// Solucion: Borrar o convertir los datos erroneos
```